

Modular Chatbot Builder Suite with Generative AI & RAG

Name: Adam Lim Han Jung

Date: 22nd April 2025

Table of Contents

1. Overview
2. Problem Statement
3. Problem Solution
4. Step 1: Upload Documents to Train the Chatbot
 - 4.1. Supported File Formats
 - 4.2. Parsing, Chunking, and Embedding Documents
 - 4.3. Storing Data in a Custom Vector Database (FAISS/Qdrant)
 - 4.4. Managing Knowledge Base for Each User
5. Step 2: Configure Chatbot Personality & Model
 - 5.1. Customize Chatbot Identity
 - 5.2. Select LLM for Generation
 - 5.3. Set Up Response Behavior
6. Step 3: Start Chatting with Your Custom Assistant
 - 6.1. Interactive Chat UI
 - 6.2. Session Management
 - 6.3. Model Switching
7. Step 4: Fine-Tune with Domain Data (Advanced)
 - 7.1. Train on Custom Data
 - 7.2. Create Domain-Specific Modes
8. Step 5: Add Tools or Plugins
 - 8.1. Extend Chatbot Functionality
 - 8.2. Plugin Architecture
9. Step 6: Deploy Chatbot Anywhere
 - 9.1. Cloud Deployment
 - 9.2. Local Deployment
10. Step 7: Save, Share, and Clone
 - 10.1. Save Configuration
 - 10.2. Share and Clone
11. Part 8: Documentation and Deployment
 - 11.1. GitHub Repository Setup

- 11.2. Detailed ReadMe Instructions
 - 11.3. Deployment on Public Website
 - 11.4. Documentation in PDF/Word Format
- 12. Conclusion
 - 12.1. Project Outcomes
 - 12.2. Future Enhancements
 - 12.3. Scalability and Future Plans
- 13. Appendix
- 14. References

Overview

Customisable chatbot, integrated with generative AI and customisable RAG.

Problem Statement

There are no available chatbots that can be integrated easily into web applications / websites.

Problem Solution

Step 1: Upload Documents to Train the Chatbot

Start with creation of the project, with the backend and the frontend. Installing and creating the following files:

Backend (FastAPI)

routes/upload.py: Handles file upload and user ID input

services/document_processing.py: Uses LangChain to parse, chunk, embed

db/vectorstore.py: Stores/retrieves chunks from FAISS or Qdrant

main.py: FastAPI entry, includes route setup and CORS

.env: Store OpenAI API keys and DB config

Frontend (React + Tailwind + Zustand)

UploadForm.tsx: Drag & drop or file input, handles form submission

ChatWindow.tsx: Chat UI with conversation history and message bubbles

api.ts: Axios wrapper to call /upload and future endpoints (e.g., /chat)

useChatStore.ts: Stores messages, user config, and session state

For the references for the backend database vectorstore,

```
curl
"https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent?
key=AIzaSyCvW9OkQ-AQuwq9jp18i4b2cl_7-uXoeDA" \
-H 'Content-Type: application/json' \
-X POST \
-d '{
  "contents": [{
    "parts":[{"text": "Explain how AI works"}]}]
  }'
}
```

```
.venv\jung@Adams-MacBook-Air backend % curl -X POST "http://127.0.0.1:8000/api/upload"
\ -H "accept: application/json" \ -H "Content-Type: multipart/form-data" \
-F "file=@/Users/jung/Desktop/Chatbot/build-your-cbot/backend/app/test/test.pdf"
```

Step 2: Configure Chatbot Personality & Model

Customize Chatbot Identity

Define the chatbot's name, greeting message, and personality.

Set its role (e.g., customer support, personal assistant, academic mentor) and desired tone (casual, formal, humorous).

Select LLM for Generation

Choose from multiple LLMs, such as GPT-4, Claude, Mistral, or DeepSeek.

Optionally, configure model parameters like temperature, top_p, and max tokens to control the creativity and response length.

Set Up Response Behavior

Configure how the chatbot should respond to various types of queries.

Fine-tune behavior for different types of interactions (e.g., troubleshooting, casual conversation, technical support).

Step 3: Start Chatting with Your Custom Assistant

Interactive Chat UI

The chatbot UI is built with React and provides an intuitive interface for real-time conversations.

Includes features like typing effects, message history, and quick replies.

Session Management

Each session is saved and can be reviewed later.

Option to reset chat history and start new sessions.

Model Switching

Users can switch between different LLMs during a conversation for varied responses.

Test different models in real-time to find the one that suits the chatbot's purpose best.

Step 4: Fine-Tune with Domain Data (Advanced)

Train on Custom Data

Fine-tune the chatbot using domain-specific datasets (e.g., product knowledge, company FAQs).

Upload training data in JSONL format to fine-tune models like GPT-3 or DeepSeek.

Create Domain-Specific Modes

Develop specialized modes for specific tasks (e.g., sales, support, education).

Each mode can have its own set of behaviors, responses, and document sources.

Step 5: Add Tools or Plugins

Extend Chatbot Functionality

Integrate tools such as a Python REPL for calculations or code execution.

Add API integrations like Google Calendar, Google Search, or SQL databases for dynamic queries.

Plugin Architecture

The modular architecture allows users to easily add new tools and plugins as needed.

Popular tools include Google Search, Python functions, and advanced math solvers.

Step 6: Deploy Chatbot Anywhere

Cloud Deployment

Deploy your chatbot to platforms like Vercel, Netlify, or Railway with simple deployment configurations.

Host the backend on AWS, Render, or Google Cloud for scalability.

Local Deployment

For self-hosted solutions, use Docker containers or FastAPI for a local backend.

Deploy chatbots on custom domains or integrate them into existing websites via embeddable widgets.

Step 7: Save, Share, and Clone

Save Configuration

Save your chatbot setup as a configuration file (.json or .yaml) to preserve customizations.

Easily export the configuration for backup or reuse.

Share and Clone

Share your chatbot by generating a link or by cloning it to other platforms (e.g., GitHub).

Clone your chatbot for different use cases or clients, while retaining all customizations.

Step 8: Documentation & Developer Mode

Access Codebase and Customization Options

Developers can modify prompt templates, add new plugins, and extend the bot's capabilities.

GitHub Repository for code contributions and community-driven enhancements.

Advanced Developer Tools

Modify backend functionality or integrate new vector databases (FAISS, Weaviate).

Configure new embedding models or chain functions with LangChain.

Core Features

Customizable Chatbot Personality

Multi-LLM Support (GPT, Claude, Mistral)

Document Upload and Parsing

Real-Time Chat Interaction

Model Fine-Tuning

Modular Tool and Plugin System

Easy Deployment (Cloud & Local)

Save, Share, and Clone Functionality

Developer Mode for Advanced Customization

Tech Stack:

Frontend: React, TailwindCSS, Zustand

Backend: FastAPI, AWS Lambda, API Gateway

AI Models: OpenAI, DeepSeek, Claude, Mistral, LangChain

Database: FAISS, Qdrant, DynamoDB (for chat history)

Deployment: Docker, AWS, Vercel, Railway

1. Overview

This is an AGV Simulation and Planning Software, integrated with AI Chatbot for users new to warehouse automation. Most pre-sales engineers in the warehouse automation industry lack the resources to AGV simulation when trying to meet the customer's requirements such as throughput, budget, size of space, dimensions of goods stored, and their WMS (Warehouse Management System) data format, therefore this AGV simulation, consists of a Warehouse Automation Calculator, which allows users to input the necessary details such as size of warehouse automation area, the SKU order sizes in carton, the weight of goods, the working hours, and more, which is then calculated dynamically to determine the number of robots, shelves, stations, and more to achieve the desired throughput. Then, user can use the UI provided to map out the AGV area to test the simulation, using a Standard Interface for sending orders for picking / putaway through the API endpoints which is connected to MongoDB databases and collections to store, retrieve and view the orders created being fulfilled in real time, and filter through the tables to view the WMS database structure to integrate with this simulation if needed in the future. Also there is a chatbot interface for answering any queries about the software and warehouse automation in general ...